

```

1 import numpy as np
2
3
4 class TrueTest:
5     totalClients = 1000
6     totalQuestions = 1000
7
8     canBeImposterFactor = 0.2
9     wantParticipateFactor = 0.7
10    canGiveWrongFactor = 0.2
11    questionAnswerFactor = 0.4
12    minGoodClientsFactor = 0.6
13    randomValidateFactor = 0.3
14
15    sleeperFactor = 0.2
16    sleeperStartFactor = 0.7
17
18    MIN_RATING = 0.1
19    MAX_RATING = 0.9
20
21    @classmethod
22    def run(cls, seed=None):
23        rng = np.random.default_rng(seed)
24
25        n = cls.totalClients
26        q_count = cls.totalQuestions
27
28        ratings = np.full(n, 0.5, dtype=np.float64)
29
30        imposters = rng.random(n) < cls.canBeImposterFactor
31        sleepers = (
32            imposters &
33            (rng.random(n) < cls.sleeperFactor)
34        )
35
36        questions = (
37            rng.random(q_count) < cls.questionAnswerFactor
38        )
39
40        good_count = 0
41        validation_count = 0
42
43        sleeper_start_q = max(
44            1,
45            int(q_count * cls.sleeperStartFactor)
46        )
47
48        for q, correct_answer in enumerate(questions, start=1):
49            participate = (
50                rng.random(n) < cls.wantParticipateFactor
51            )
52
53            if not np.any(participate):
54                continue
55
56            pidx = np.flatnonzero(participate)
57
58            malicious = imposters & (
59                ~sleepers | (q >= sleeper_start_q)
60            )
61
62            malicious_part = malicious[pidx]
63            answers = np.empty(pidx.size, dtype=bool)
64
65            malicious_count = malicious_part.sum()
66            good_part = ~malicious_part
67            good_part_count = good_part.sum()
68
69            if malicious_count:
70                answers[malicious_part] = (
71                    rng.random(malicious_count) < 0.5
72                )
73
74            if good_part_count:
75                good_answers = np.full(
76                    good_part_count,
77                    correct_answer,
78                    dtype=bool
79                )
80
81            if cls.canGiveWrongFactor > 0:

```

```

81 .....
82 ..... flip = (
83 .....     rng.random(good_part_count)
84 .....     < cls.canGiveWrongFactor
85 ..... )
86 ..... good_answers ^= flip
87 .....
88 ..... answers[good_part] = good_answers
89 .....
90 ..... pratings = ratings[pidx]
91 .....
92 ..... yes_score = pratings[answers].sum()
93 ..... no_score = pratings[~answers].sum()
94 .....
95 ..... enough_good = (
96 .....     good_count >=
97 .....     cls.minGoodClientsFactor * n
98 ..... )
99 .....
100 ..... random_validate = (
101 .....     rng.random() < cls.randomValidateFactor
102 ..... )
103 .....
104 ..... tied = yes_score == no_score
105 .....
106 ..... validate = (
107 .....     not enough_good
108 .....     or random_validate
109 .....     or tied
110 ..... )
111 .....
112 ..... if validate:
113 .....     validation_count += 1
114 .....     winning_answer = correct_answer
115 ..... else:
116 .....     winning_answer = yes_score > no_score
117 .....
118 ..... total_score = yes_score + no_score
119 .....
120 ..... if total_score == 0:
121 .....     continue
122 .....
123 ..... winning_score = (
124 .....     yes_score
125 .....     if winning_answer
126 .....     else no_score
127 ..... )
128 .....
129 ..... consensus = winning_score / total_score
130 ..... strength = 2.0 * (consensus - 0.5)
131 ..... update = 0.05 + (0.10 * strength)
132 .....
133 ..... new_ratings = pratings.copy()
134 ..... correct_side = answers == winning_answer
135 .....
136 ..... old_good = pratings > 0.5
137 .....
138 ..... if np.any(correct_side):
139 .....     new_ratings[correct_side] += (
140 .....         update *
141 .....         (cls.MAX_RATING - pratings[correct_side])
142 .....     )
143 .....
144 ..... if np.any(~correct_side):
145 .....     new_ratings[~correct_side] -= (
146 .....         update *
147 .....         (pratings[~correct_side] - cls.MIN_RATING)
148 .....     )
149 .....
150 ..... new_good = new_ratings > 0.5
151 .....
152 ..... good_count += (
153 .....     np.count_nonzero(~old_good & new_good)
154 .....     - np.count_nonzero(old_good & ~new_good)
155 ..... )
156 .....
157 ..... ratings[pidx] = new_ratings
158 .....
159 ..... cls.display(
160 .....     ratings,
161 .....     imposters,
162 .....     sleepers,
163 .....     validation_count

```

```

164         )
165
166     @classmethod
167     def display(
168         cls,
169         ratings,
170         imposters,
171         sleepers,
172         validation_count
173     ):
174         n = len(ratings)
175
176         window_size = max(
177             1,
178             int(n * cls.canBeImposterFactor)
179         )
180
181         window_size = min(
182             window_size,
183             n // 2
184         )
185
186         bottom_idx = np.argpartition(
187             ratings,
188             window_size - 1
189        )[:window_size]
190
191         top_idx = np.argpartition(
192             ratings,
193             n - window_size
194        )[-window_size:]
195
196         total_imposters = int(imposters.sum())
197         total_good = n - total_imposters
198         total_sleepers = int(sleepers.sum())
199
200         bottom_imposters = int(imposters[bottom_idx].sum())
201         top_imposters = int(imposters[top_idx].sum())
202
203         bottom_good = window_size - bottom_imposters
204         top_good = window_size - top_imposters
205
206         bottom_sleepers = int(sleepers[bottom_idx].sum())
207         top_sleepers = int(sleepers[top_idx].sum())
208
209         bottom_imposter_ratings = ratings[
210             bottom_idx[imposters[bottom_idx]]
211         ]
212
213         top_imposter_ratings = ratings[
214             top_idx[imposters[top_idx]]
215         ]
216
217         imposter_ratings = ratings[imposters]
218         good_ratings = ratings[~imposters]
219         sleeper_ratings = ratings[sleepers]
220
221         bottom_avg = ratings[bottom_idx].mean()
222         top_avg = ratings[top_idx].mean()
223
224         avg_imposter = (
225             imposter_ratings.mean()
226             if total_imposters
227             else 0.0
228         )
229
230         avg_good = (
231             good_ratings.mean()
232             if total_good
233             else 0.0
234         )
235
236         avg_sleeper = (
237             sleeper_ratings.mean()
238             if total_sleepers
239             else 0.0
240         )
241
242         avg_bottom_imposter = (
243             bottom_imposter_ratings.mean()
244             if bottom_imposter_ratings.size
245             else 0.0
246         )

```

```

247
248     avg_top_imposter = (
249         top_imposter_ratings.mean()
250         if top_imposter_ratings.size
251         else 0.0
252     )
253
254     validation_saved = cls.totalQuestions - validation_count
255     validation_rate = (
256         validation_count / cls.totalQuestions * 100
257     )
258
259     bottom_imposter_share = (
260         bottom_imposters / window_size * 100
261     )
262
263     top_imposter_share = (
264         top_imposters / window_size * 100
265     )
266
267     bottom_good_share = (
268         bottom_good / window_size * 100
269     )
270
271     top_good_share = (
272         top_good / window_size * 100
273     )
274
275     imposters_in_bottom_rate = (
276         bottom_imposters / total_imposters * 100
277         if total_imposters
278         else 0.0
279     )
280
281     imposters_in_top_rate = (
282         top_imposters / total_imposters * 100
283         if total_imposters
284         else 0.0
285     )
286
287     good_in_bottom_rate = (
288         bottom_good / total_good * 100
289         if total_good
290         else 0.0
291     )
292
293     good_in_top_rate = (
294         top_good / total_good * 100
295         if total_good
296         else 0.0
297     )
298
299     avg_score_gap = avg_good - avg_imposter
300
301     print("\n" + "=" * 60)
302     print("FINAL RESULTS")
303     print("=" * 60)
304
305     print(f"Total Clients: {total_clients}")
306     print(f"Total Imposters: {total_imposters}")
307     print(f"Total Good Clients: {total_good}")
308     print(f"Total Sleepers: {total_sleepers}")
309
310     print("\nVALIDATION")
311     print("-" * 60)
312     print(f"Total Questions: {cls.totalQuestions}")
313     print(f"Total Tests Needed: {validation_count}")
314     print(f"Validation Rate: {validation_rate:.2f}%")
315     print(f"Tests Saved: {validation_saved}")
316     print(
317         f"Tests Saved Percentage: {validation_saved / validation_count * 100:.2f}%"
318         f"{100 - validation_rate:.2f}%"
319     )
320
321     print("\nBOTTOM WINDOW")
322     print("-" * 60)
323     print(f"Window Size: {window_size}")
324     print(f"Imposters in Bottom: {bottom_imposters}")
325     print(
326         f"Imposters in Bottom %: {bottom_imposter_share:.2f}%"
327         f"{top_imposter_share:.2f}%"
328     )
329     print(

```

```

330         print(
331             f"All Imposters in Bottom %: "
332             f"{imposters_in_bottom_rate:.2f}%"
333         )
334         print(f"Good Clients in Bottom: {bottom_good}")
335         print(
336             f"Good Share of Bottom: "
337             f"{bottom_good_share:.2f}%"
338         )
339         print(
340             f"Good Clients Lost Bottom %: "
341             f"{good_in_bottom_rate:.2f}%"
342         )
343         print(f"Sleepers in Bottom: {bottom_sleepers}")
344         print(
345             f"Average Bottom Score: "
346             f"{bottom_avg:.4f}"
347         )
348         print(
349             f"Average Bottom Imposter: "
350             f"{avg_bottom_imposter:.4f}"
351         )
352         print("\nTOP WINDOW")
353         print("-" * 60)
354         print(f"Window Size: {window_size}")
355         print(f"Imposters in Top: {top_imposters}")
356         print(
357             f"Imposters in Top %: "
358             f"{top_imposter_share:.2f}%"
359         )
360         print(
361             f"All Imposters in Top %: "
362             f"{imposters_in_top_rate:.2f}%"
363         )
364         print(f"Good Clients in Top: {top_good}")
365         print(
366             f"Good Share of Top: "
367             f"{top_good_share:.2f}%"
368         )
369         print(
370             f"Good Clients in Top %: "
371             f"{good_in_top_rate:.2f}%"
372         )
373         print(f"Sleepers in Top: {top_sleepers}")
374         print(
375             f"Average Top Score: "
376             f"{top_avg:.4f}"
377         )
378         print(
379             f"Average Top Imposter: "
380             f"{avg_top_imposter:.4f}"
381         )
382         print("\nREPUTATION")
383         print("-" * 60)
384         print(
385             f"Average Imposter Score: "
386             f"{avg_imposter:.4f}"
387         )
388         print(
389             f"Average Good Score: "
390             f"{avg_good:.4f}"
391         )
392         print(
393             f"Good - Imposter Gap: "
394             f"{avg_score_gap:.4f}"
395         )
396         if total_sleepers:
397             print(
398                 f"Average Sleeper Score: "
399                 f"{avg_sleeper:.4f}"
400             )
401         print("-" * 60)
402     if __name__ == "__main__":
403         TrueTest.run(seed=None)
404

```

```
=====
FINAL RESULTS
=====
Total Clients:          1000
Total Imposters:        192
Total Good Clients:     808
Total Sleepers:         31

VALIDATION
-----
Total Questions:       1000
Total Tests Needed:    306
Validation Rate:       30.60%
Tests Saved:          694
Tests Saved Percentage: 69.40%

BOTTOM WINDOW
-----
Window Size:           200
Imposters in Bottom:   167
Imposters in Bottom %: 83.50%
All Imposters in Bottom %: 86.98%
Good Clients in Bottom: 33
Good Share of Bottom:  16.50%
Good Clients Lost Bottom %: 4.08%
Sleepers in Bottom:    29
Average Bottom Score:  0.4890
Average Bottom Imposter: 0.4756

TOP WINDOW
-----
Window Size:           200
Imposters in Top:      0
Imposters in Top %:    0.00%
All Imposters in Top %: 0.00%
Good Clients in Top:   200
Good Share of Top:     100.00%
Good Clients in Top %: 24.75%
Sleepers in Top:       0
Average Top Score:     0.8274
Average Top Imposter:  0.0000

REPUTATION
-----
Average Imposter Score: 0.4963
Average Good Score:     0.7361
Good - Imposter Gap:    0.2398
Average Sleeper Score:  0.4865
=====
```

1 Start coding or [generate](#) with AI.